

NuBasic

A Primitive Operating System by Ryan Harding

Based on a Project by Khan Shaikhul Hadi

Github Repo:

Introduction

For this project wanted to explore how to make an operating system that interacts with the hardware on a level beyond simply recognizing it. To that end I based my project on the existing “Personal OS” by Khan Shaikhul Hadi, which contained an effective bootloader, CPU interrupt handling, and screen and keyboard drivers. My work primarily consisted of integrating three brand new elements to the OS: Memory management, an interpreter to take commands typed in by the user and execute them, and a proper shell prompt to type commands at.

The main goal was to create something approaching the operating system used in early computers, BASIC, and thus understand how operating systems became something users could actually interact with.

Methodology

The Base System

The project was done on a mid-2010s Dell Laptop running Ubuntu 20.04, using gcc 4.9.1 and binutils 2.24 for C cross-compilation and NASM for the Assembly bootloader. The OS is run by compiling a custom disk image for QEMU emulating an i386 machine with 128 MB of RAM.

The bootloader and associated elements were programmed in assembly by Hadi in the first iteration of the project, everything else was done in C. C’s default standard libraries were not used, instead Hadi implemented custom libraries and functions that I expanded on.

Foundational Improvements

In order to move the OS away from just outputting characters to the screen, I rewrote the Keyboard driver to pass characters into a buffer rather than just printing immediately, with printing to be handled by the shell. I also added a single-character version of printing to the

Screen driver. I added several functions to the utility libraries, including string length, comparison, and copying, as well as conversions between integers, strings, and hexadecimal values.

Memory Implementation

My implementation of memory is a heap built on top of a simple bitmap. Right now, I have a set block size of 4096 bytes *and* a set 1024 blocks, meaning addressable memory is hard locked at 4 MB, which is still far more memory than systems using BASIC would've been able to recognize. The basic bitmap allocation simply searches the map and finds the first 0 "free" bit and sets it to 1, returning the address of the block that index corresponds to.

The heap implementation on top (To allow for allocating out-of-order) of that uses a linked list with a very rudimentary version of "malloc". It simply iterates through the linked list checking the free bit of every item and returning the first block with a free bit or allocating a new block and adding it to the linked list. There is no spreading of data across multiple blocks, so there is a hard limit to data stored for a single variable or array as well as a large risk of internal fragmentation.

"Free" algorithms, but on a base level and for the heap, are implemented, but due to the lack of any sort of garbage collection or manual free command in the interpreter, they go un-used. There is also some leftover AI code from when I was experimenting with proper processes and loading them into memory, but I dropped that early on.

Interpreter Design

The interpreter takes an input string from the shell after a line break and compares it to known commands to accept. If a command is unknown, it outputs an error. After an error or most commands, it returns an exit flag of "0" to the shell, indicating execution should continue. The interpreter currently supports Peeks and Pokes, Variables and Arrays, Printing, Clearing the Screen, and resetting or shutting down the machine.

Peeks and Pokes

Both "PEEK" and "POKE" take the first argument as an address and determine if it's decimal or hexadecimal based on if the first two characters are "0X" or not. It then calls a library function to translate the ascii value into a proper address depending on that. "PEEK" will then get the value of the memory block at that address, convert it to an ascii string (As a decimal value), and print that value.

“POKE” will instead take a second argument, in either decimal or hex, convert it to a proper integer and store it at the memory block from the first argument. It will then return a success message to the screen.

Variables and Arrays

The “LET” command uses a syntax of “LET X = val”, where “X” is a single-character variable name. Every variable or array is addressed in a single array consisting of structs with the name, type (Integer or Array), data address, and size (1 for variables, more for arrays). A function is called to seek the array for an existing variable with that name, and if one is found the variable and its memory block is reused and updated. Otherwise, a new integer variable is placed in the array and a new memory block is allocated. Both variables and arrays only support decimal values.

To define an array, the command “ARRAY X size” is used, with “X” once again being a single-character identifier. A new variable, with the type set to array and size set to the input integer size value, is placed in the buffer, and the appropriate memory blocks are allocated and initialized to 0.

Storing values in an array is handled with “SET X index value”, which finds the array “X” in memory, and sets the value at index “index” to “value”. “GET X index” prints the value at “index” of array “X” to the screen using similar measures, searching the global symbol table for “X” and then getting the data address from the symbol. Both “SET” and “GET” may return errors for improper syntax or if the identifier used is a variable rather than an array.

Printing

“PRINT” works in two ways. If the character after the command and a space is a single or double quote, it will read and print every character after the quote until a closing quote is reached and return. Otherwise, it takes a variable identifier, seeks the variable in the table, and prints its value from the memory address.

Clear, Reset, and Shutdown

The command “CLEAR” simply calls a function in the screen driver to flush the screen.

“RESET” returns an exit flag of 2 to the shell, which tells it to exit the shell but tells the kernel to loop back and initialize the system and memory from scratch again.

“SHUTDOWN” returns an exit flag of 1 to the shell, which tells it to return a flag to the kernel that exits the loop and sends the ACPI shutdown command that causes QEMU to shut down.

Shell Implementation

The Kernel calls into the shell, which seeks characters from the buffer in the keyboard driver and moves them into an input buffer. A backspace will trigger a reversal of one character and clear it from the screen. Every proper character is printed to the screen using the new function in the screen driver. Enter triggers a line break and returns from the seek loop, and then the finished buffer (the typed command) is passed to the interpreter. It also takes the exit flag returned by the interpreter and either goes back to seek the next command or signals a shutdown or reset to the kernel.

Results

Obstacles / Goals Dropped

Early on I faced some confusion as to exactly what I wanted from this project. I started hoping I could make something much more complex, before scaling way back to essentially a glorified notepad that could save and recall text. Eventually I settled on executing code again, but in a much simpler way.

In execution, I faced obstacles largely with the compilation process and actually getting it up and running. I needed to use a secondary system from my primary computer (An M2 MacBook Pro) because it wouldn't work on Apple Silicon, and my first x86 laptop was unreliable. I also faced some issues getting a new installation of VSCode to be compatible with my version control on a new Ubuntu installation on the Dell, as well as facing driver issues in general.

Over time I dropped goals of storing and recalling text notes from memory, Function Key usage, and proper process implementation (Which is still orphaned in the code as a weird AI code experiment). I had also wanted to have a more extensive implementation of memory at the start with protection, but this was not only unfeasible but also would've been going beyond the early operating systems I was aiming to approximate.

Goals Accomplished

I was successful in implementing a proper code interpreter that handles syntax and provides the appropriate result for every supported command. I implemented a simple and inefficient but still usable memory system that accurately stores and recalls user-input values in memory. I enabled proper reset and shutdown handling in the kernel to allow the user to get out of the OS without forcibly closing QEMU. While it's an extremely simple operating system, I was successful in approaching something BASIC-like. As there's no

benchmarks or performance measures, I can only point to my in-class demo and the code itself for verifiable results.

Lessons

The biggest takeaway I had was that I realized just how simple these early operating systems were and how piecemeal development is, as well as how important comprehensive driver interaction with the kernel/shell is. A good chunk of the project was rewriting the keyboard driver, which made the shell trivial to implement. Primitive memory management also wasn't nearly as complicated as I had thought (Especially when early AI experiments kept trying to push me to paging immediately, hence why I abandoned it). It's just a matter of tracking free bits and segmenting what C can already recognize and address.

Of course, that's also where I'm much more privileged than the early OS developers I'm trying to follow. These early OS' weren't written in easy C files, everything was chunks of assembly, and bootloaders weren't as standardized either. Microsoft BASIC was written in 8080 machine languages, which was a much more complicated environment. Yet because of that, the entire language/operating system fit in 4K of RAM, which is something my OS doesn't do. In that way there's certainly a trade-off for the simpler, more comprehensive environment compared to the machine code.

Conclusion

Ultimately, I was successful in, and greatly enjoyed, making a primitive operating system environment, and would like to continue work on this for my own learning. I was able to put what I had learned about memory management in this class to use as well as dig into my experiences with computer architecture and compiler construction when making the interpreter. It was also just fun to try to recreate and put my own spin on BASIC, since it was tinkering with Apple II emulators and such that made me interested in programming to begin with. Knowing that these original developers managed to make something even more complex with a fraction of the tools also put in perspective how talented they were and gave me a newfound respect for them.

Future Work

The first next step would be to expand and improve the interpreter. Add a proper tokenizer to streamline the parsing and add more commands and improve the handling of variables and arrays (E.g. multi-character identifiers and multi-dimensional arrays). Then would be to implement some sort of “Begin-End” format to enable the creation of multi-line programs and execute as a process, something I already have some experiments related to. Then would come improving the memory management with protection and reducing fragmentation. Polishing up some leftover code and making compilation easier is also a good next step.